

1. Programmatic Navigation in React Router

Programmatic navigation in React Router refers to navigating between routes using JavaScript code instead of clicking links. It allows you to redirect users automatically after certain actions like form submission, login, or logout.

useHistory (React Router v5)

In React Router v5, the useHistory hook is used for navigation. It provides access to the history object with methods like: • history.push('/path') → Navigate to a new route. • history.replace('/path') → Replace the current route in history.

Example (useHistory):

```
import { useHistory } from 'react-router-dom'; function Login() { const history = useHistory(); const handleLogin = () => { history.push('/dashboard'); }; return Login; }
```

useNavigate (React Router v6+)

In React Router v6, useNavigate replaced useHistory. It returns a function for navigation, simplifying route changes.

Example (useNavigate):

```
import { useNavigate } from 'react-router-dom'; function Login() { const navigate = useNavigate(); const handleLogin = () => { navigate('/dashboard'); }; return Login; }
```

When to Use Which:

• useHistory → Used in React Router v5 (older projects) • useNavigate → Used in React Router v6+ (newer projects)

2. useParams Hook

The useParams hook in React Router allows you to access dynamic route parameters from the URL. It is useful when a route contains variable segments, like user IDs or product IDs.

Example:

```
} /> import { useParams } from 'react-router-dom'; function UserProfile() { const { id } = useParams(); return User ID: {id}; }
```

Scenario:

If you have a list of users at /users, clicking on a specific user (like /users/101) navigates to a page showing that user's details. The useParams hook extracts the dynamic ID (101) from the URL so the component can fetch and display the correct user data.